

Optimización de código: concepto y tipos

Local · Global · Independiente · Dependiente de la máquina

Idea clave: qué es la optimización

La optimización de código consiste en mejorar un programa para que vaya más rápido o consuma menos recursos, sin cambiar lo que hace.

El programa hace exactamente lo mismo que antes — mismos resultados, misma funcionalidad — pero lo hace mejor: evita trabajo innecesario, repite menos operaciones o usa los recursos con más cuidado.

Optimizar $\neq$ cambiar la lógica

Si tras «optimizar» el programa devuelve resultados distintos, no has optimizado: has cambiado el comportamiento. Eso es un cambio de lógica, no una mejora de rendimiento.

✗ Cambia el resultado
→ No es optimización,
es cambio de lógica

✓ Mismo resultado, mejor rendimiento
→ Esto sí es optimización

¿Cuándo vale la pena optimizar?

No todo código necesita optimización. Hacerlo antes de tiempo puede dificultar la lectura sin ningún beneficio. La pregunta clave: ¿hay un problema real que los usuarios notan?

Vale la pena

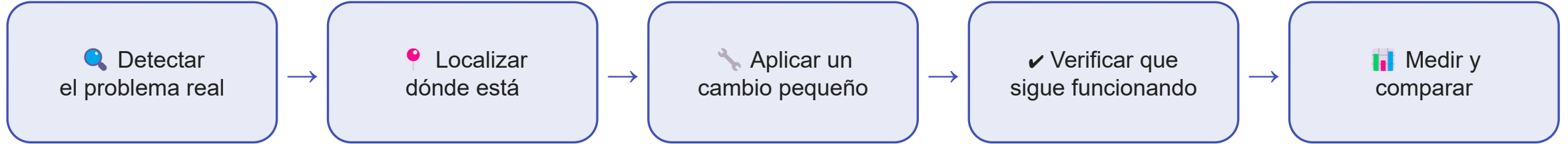
- El programa tarda demasiado y los usuarios lo notan
- Hay operaciones que se repiten miles de veces (bucles grandes, búsquedas frecuentes)
- Se usan más recursos de los necesarios (memoria, consultas a la BD)

No vale la pena

- El programa ya va bien y los usuarios no se quejan
- La «mejora» haría el código mucho más difícil de entender
- No has medido primero: sin medir, no sabes si hay problema

Mide antes de cambiar: sin medición, optimizar es una suposición. Ejecuta el código, anota el tiempo, aplica el cambio y vuelve a medir.

Proceso recomendado de optimización



Por qué importa cada paso

- **Detectar y localizar**
 - El problema no siempre está donde parece. Un programa lento puede tener el cuello de botella en un único método.
- **Un cambio pequeño**
 - No cambies varias cosas a la vez. Si cambias tres y mejora, no sabes cuál fue la responsable.
- **Verificar**
 - Una optimización que rompe algo no vale nada.
- **Medir y comparar**
 - Si no hay diferencia en los números, el cambio no ha servido.

Esto es exactamente lo que harás en la Actividad 4.1: mides, cambias una cosa, vuelves a medir.

Tipos de optimización: clasificación

Las optimizaciones se clasifican según dos preguntas independientes entre sí. La mayoría de las que escribes en código son independientes de la máquina.

Pregunta	Respuesta A	Respuesta B
¿Qué parte del código afecta?	Solo un fragmento → Local	El sistema como conjunto → Global
¿Depende del entorno donde corre?	No → Independiente de la máquina	Sí → Dependiente de la máquina

Importante: son dos ejes distintos

- Una optimización puede ser local e independiente (lo más habitual en el día a día).
- O global e independiente (por ejemplo, paginación: afecta al diseño del sistema, pero el beneficio está en el código).
- Las dependientes de la máquina son territorio de administración de sistemas, no de desarrollo.

Optimización local: salida temprana

Una optimización local mejora un fragmento concreto (un método, un bucle) sin cambiar la estructura general. Ejemplo: buscas el primer administrador en una lista de 50.000 roles. Está en la posición 3.

Sin optimizar

```
boolean encontrado = false;
for (String rol : roles) {
    if ("ADMIN".equals(rol)) {
        encontrado = true;
        // sigue con 49.997 más...
    }
}
return encontrado;
```

Optimizado

```
for (String rol : roles) {
    if ("ADMIN".equals(rol)) {
        // sale en cuanto lo encuentra
        return true;
    }
}
return false;
```

El resultado es idéntico. La diferencia: sin optimizar el bucle hace 50.000 iteraciones. Optimizado, hace 3.

Optimización local: cerrar recursos

Cuando abres un fichero o una conexión y no la cierras, el sistema sigue reservando memoria para ella aunque ya no la uses. Esto se acumula y acaba ralentizando el programa en ejecuciones largas.

Solución: try-with-resources

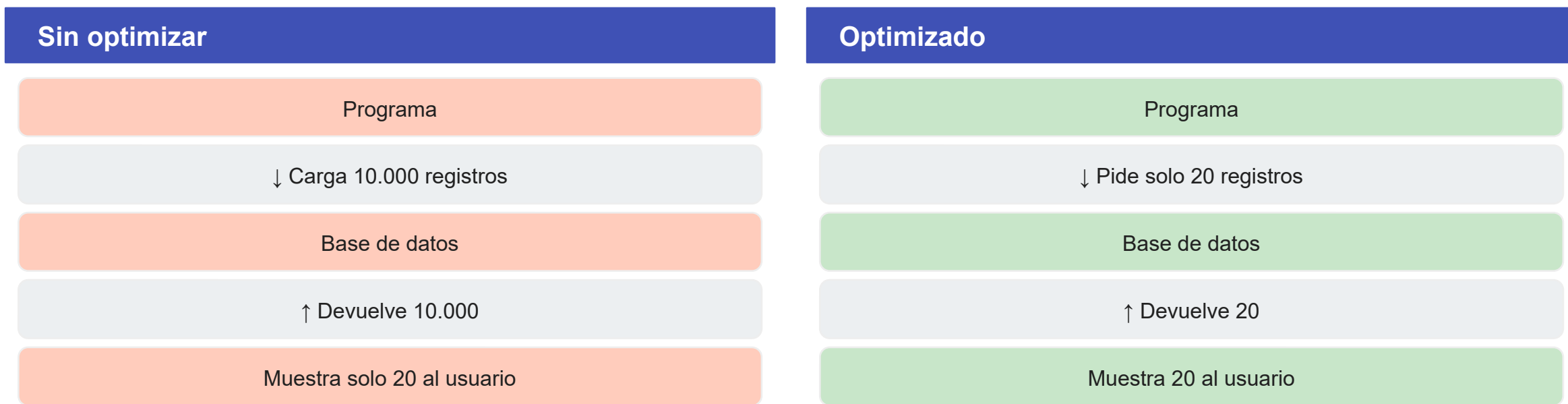
```
public static int contarLineas(String ruta) throws IOException {  
    int contador = 0;  
    // try-with-resources cierra el fichero automáticamente al salir  
    // del bloque, aunque se produzca un error durante la lectura  
    try (BufferedReader br =  
        new BufferedReader(new FileReader(ruta))) {  
        while (br.readLine() != null) {  
            contador++;  
        }  
    }  
    return contador;  
}
```

El resultado no cambia. Lo que cambia es que el programa no acumula recursos abiertos que eventualmente provocarían un fallo.

Optimización global: paginación

Una optimización global cambia el diseño de cómo el programa accede a los datos. La señal más habitual: el programa hace mucho trabajo antes de mostrar cualquier resultado.

Situación: tienes 10.000 productos y el usuario solo ve 20 en pantalla.



La mejora consiste en pedirle al origen de datos solo los elementos que se van a mostrar. El resultado para el usuario es el mismo; el trabajo del programa es radicalmente distinto.

Optimización global: caché

Situación: un método devuelve la lista de provincias de España. Esa lista no cambia nunca. Si la consultas a la base de datos cada vez que alguien abre la pantalla, estás repitiendo trabajo innecesario.

Solución: caché

Guarda el resultado la primera vez y devuélvelo directamente en las siguientes peticiones, sin volver a consultar. A esto se le llama caché: una copia temporal de un resultado ya calculado.

1ª petición

- consulta la BD
- guarda en caché
- devuelve resultado

Siguientes peticiones

- devuelve desde caché
- sin consultar la BD
- mucho más rápido

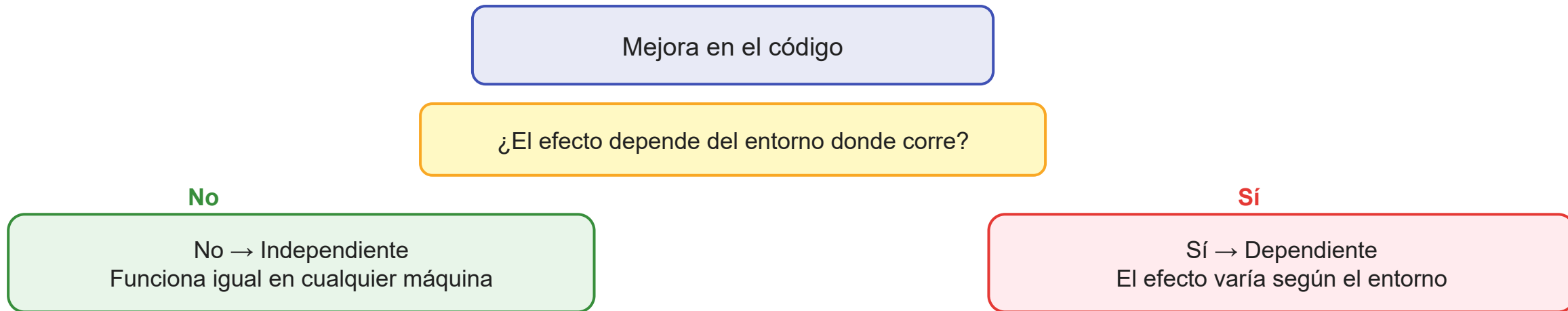
La caché tiene trampa: funciona bien cuando los datos cambian poco o nunca. Si cambian con frecuencia, necesitas decidir cuándo la copia queda obsoleta. Usarla sin pensar puede mostrar información desactualizada.

Optimización independiente de la máquina

Responde a: ¿el efecto de la mejora depende de la máquina donde corre el programa? Una optimización es independiente cuando la mejora está en el código, no en el hardware.

Si cambias la lógica de un bucle, añades una salida temprana o introduces paginación, el beneficio funciona igual en un portátil de clase que en un servidor de producción con 128 GB de RAM.

Esquema de clasificación



Todos los ejemplos anteriores (salida temprana, try-with-resources, paginación, caché) son independientes de la máquina.

Optimización dependiente de la máquina

Una optimización es dependiente cuando la mejora no está en el código sino en el entorno donde corre: más memoria, mejor configuración del servidor, ajustes del sistema operativo.

Memoria de la JVM

- La JVM (Java Virtual Machine) tiene asignada una cantidad máxima de memoria.
- Cuando se acerca al límite, el recolector de basura se activa con mucha frecuencia y ralentiza el programa.
- Aumentar esa memoria mejora el rendimiento sin tocar una sola línea de código.
- El efecto depende de cuánta memoria tenga la máquina.

Compilador JIT

- El compilador JIT (Just-In-Time) detecta qué métodos se usan más y los optimiza sobre la marcha.
- Las primeras ejecuciones son más lentas: el JIT todavía no ha optimizado nada.
- A partir de la segunda o tercera ejecución, el rendimiento mejora notablemente.
- Si mides tiempos, las primeras mediciones pueden ser menos representativas.

Antes de cambiar la configuración del servidor o pedir más hardware, agota primero las mejoras de código. Suelen tener más impacto, no cuestan dinero y funcionan en cualquier entorno.

Resumen de los tipos de optimización

Tipo	Alcance	Depende del entorno	Ejemplos
Local	Un fragmento concreto	No	Salida temprana, cerrar recursos, evitar recálculo
Global	El sistema como conjunto	No	Paginación, caché, reducir datos cargados
Independiente	Local o global	No	Cualquier mejora que está en el código, no en el servidor
Dependiente	Infraestructura	Sí	Más memoria JVM, configurar servidor, ajustar contenedor

Lo más habitual en la práctica: optimizaciones locales e independientes. Mejoras pequeñas en el código que funcionan en cualquier entorno. Las globales aparecen cuando el diseño del acceso a datos es el problema. Las dependientes de la máquina son territorio de sistemas, no de desarrollo.